

Evaluating High-Dimensional Data Structures for RGB Image Similarity Search: LSH and M-Tree vs. Classical Approaches

Luan Barbosa Rosa Carrieros   [Pontifical Catholic University of Minas Gerais | luan.rosa@sga.pucminas.br]

Diego Moreira Rocha  [Pontifical Catholic University of Minas Gerais | diego.moreira@sga.pucminas.br]

Iago Fereguetti Ribeiro  [Pontifical Catholic University of Minas Gerais | iago.fereguetti@sga.pucminas.br]

Bernardo Ferreira Temponi  [Pontifical Catholic University of Minas Gerais | bernardo.temponi@sga.pucminas.br]

Arthur Gonçalves de Moraes  [Pontifical Catholic University of Minas Gerais | arthur.moraes@sga.pucminas.br]

 PUC Minas, Instituto de Ciências Exatas e Informática (ICEI), Av. Dom José Gaspar, 500, Coração Eucarístico, Belo Horizonte, MG, 30535-901, Brazil.

Abstract.

We evaluate similarity search structures across two datasets: ColorHistogram 32D (7) (68,040 vectors) and RGB 3D images (26,179 animals). **ColorHistogram Results:** LSH demonstrates clear advantage in 32D space—achieving 2-3× faster search than LinearSearch at 50K+ scale while maintaining high precision. Insert times scale moderately (LinearSearch: 15ms, LSH: 89ms at 50K). **RGB 3D Results:** Classical structures dominate (Octree: 2.12ms search, 100% precision vs LSH: 4.62ms, 95.9% recall). This reveals *bidirectional curse of dimensionality*—high-dim structures excel in true high-dimensional spaces (32D) but impose overhead in low-dimensional contexts (RGB 3D), demonstrating critical importance of algorithm-dimensionality matching.

Keywords: Image similarity; LSH; M-Tree; ColorHistogram; High-dimensional indexing; Curse of dimensionality; RGB; 32D vectors; Performance analysis.

1 Introduction

Image similarity search in RGB color space represents a fundamental problem in computer vision with applications spanning content-based image retrieval, visual search engines, and recommendation systems. While RGB space contains only three dimensions, the selection of efficient indexing structures remains an active research area as datasets scale to millions of images.

Classical spatial indexing approaches—including hash tables, octrees, and quadrees—have demonstrated consistent effectiveness in low-dimensional spaces through geometric partitioning and spatial locality exploitation. However, the emergence of sophisticated high-dimensional indexing structures raises important questions about their applicability beyond their original design context. Locality-Sensitive Hashing (LSH) (1) promises sublinear query time through probabilistic approximation using random projections, specifically designed to combat the curse of dimensionality in high-dimensional spaces ($d \geq 50$). M-Tree (2) leverages metric space properties and triangle inequality pruning to accelerate similarity search while theoretically guaranteeing exact results.

Research Question: These structures were designed for high-dimensional scenarios such as 128-dimensional SIFT descriptors or 512-dimensional deep learning embeddings. How do they perform across different dimensionalities—both in their intended high-dimensional context (32D color histograms) and in simpler low-dimensional RGB 3D space?

This study addresses this question through dual-dataset evaluation. We test three similarity search structures: LinearSearch (baseline), LSH with random projections, and M-Tree with triangle inequality pruning. **Primary experiment:**

ColorHistogram 32D dataset (7) (68,040 vectors) representing true high-dimensional similarity search. **Validation experiment:** RGB 3D from Animals-10 (26,179 images) using OpenCV extraction. This design reveals how algorithm performance depends critically on dimensionality matching.

Key Findings: LSH demonstrates clear advantages in 32D ColorHistogram space (2-3× faster search at scale), validating its design intent. However, in RGB 3D space, classical structures (Octree, Quadtree) dominate with simpler implementations achieving better performance and precision. This bidirectional pattern confirms that high-dimensional indexing structures excel in their intended context but impose unnecessary overhead when misapplied to low-dimensional problems.

2 Methodology

RGB similarity search retrieves images $I_i = (r_i, g_i, b_i)$ within Euclidean distance threshold $\tau = 40.0$ from query $q = (r_q, g_q, b_q)$: $d(q, I_i) = \sqrt{(r_q - r_i)^2 + (g_q - g_i)^2 + (b_q - b_i)^2} \leq \tau$.

2.1 Algorithm Descriptions

Classical Structures (5): Linear Search serves as the baseline with sequential $O(n)$ examination guaranteeing exact results. Hash Search employs 3D spatial grid partitioning with dynamic cell sizing (cell=30) for $O(1)$ expected insertion and $O(k \cdot \rho)$ search where k represents neighboring cells. Hash Dynamic extends this with adaptive concentric expansion from the query cell. Octree implements recursive 3D spatial tree partitioning (maxPerNode=10) with geometric

pruning, achieving $O(\log n)$ expected operations. Quadtree uses 2D iterative spatial tree (maxPerNode=25) projecting onto R,G dimensions with blue channel verified during final distance computation.

LSH (Locality-Sensitive Hashing) (1): Implements probabilistic approximate nearest neighbor search through random projections. Uses $L=10$ independent hash tables, each with $k=4$ hash functions and projection width $w=50.0$. Each hash function projects the d -dimensional input onto random directions, partitioning space into buckets. Query processing searches all L tables in parallel, retrieving candidates from matching buckets. Designed for high-dimensional spaces ($d \geq 50$) to achieve sublinear $O(n^\rho)$ query time where $\rho < 1$, trading precision for speed through probabilistic guarantees.

M-Tree (Metric Tree) (2): Implements distance-based indexing for metric spaces using triangle inequality pruning. Each node stores entries with covering radius, enabling pruning of entire subtrees during search when triangle inequality proves they cannot contain results. Node capacity=10 triggers balanced splitting. Theoretically guarantees exact search results while reducing distance computations through geometric pruning in metric space.

2.2 Experimental Setup

All structures implemented in C++17 with unified interface. The CREATE→TEST→DESTROY pattern ensures fair evaluation by reconstructing structures per test, eliminating caching artifacts.

Primary Dataset - ColorHistogram 32D: 68,040 color histogram vectors (32 dimensions) from UCI Corel Image Features dataset (7). Query uses first vector (ID=1) with threshold $\tau = 0.3$. Scales: 1K, 5K, 10K, 25K, 50K, 68K vectors. Tests three structures: LinearSearch, LSH ($L=10$, $k=8$, $w=0.1$), M-Tree (simplified).

Validation Dataset - RGB 3D: 26,179 animal photographs (3) with RGB extracted via OpenCV (4). Threshold $\tau = 40.0$, Euclidean distance. Interactive query selection from ./query/ folder. Tests all seven structures including spatial trees (Octree, Quadtree) designed for low dimensions.

Both experiments measure insertion time, search time, and precision (similar items found within threshold).

3 Results and Analysis

3.1 ColorHistogram 32D Performance - Primary Experiment

Table 1 presents performance results for ColorHistogram 32D dataset across six scales (1K to 68K vectors). The results validate LSH’s effectiveness in true high-dimensional contexts, demonstrating clear search speedup at scale while maintaining acceptable precision trade-offs.

Table 1. ColorHistogram 32D Performance (times in milliseconds, 32 dimensions)

Structure	Op.	1K	5K	10K	25K	50K	68K
LinearSearch	Insert	0.32	1.67	3.48	8.92	15.20	21.15
	Search	0.09	0.48	1.02	2.58	4.83	6.71
LSH ($L=10, k=8$)	Insert	1.89	9.54	19.37	47.82	88.73	121.40
	Search	0.04	0.21	0.43	0.98	1.67	2.35
M-Tree	Insert	0.35	1.73	3.61	9.15	16.08	22.33
	Search	0.11	0.52	1.09	2.67	4.91	6.88

Bold values indicate best performance per category. LSH achieves $2.9\times$ search speedup at 50K scale (1.67ms vs 4.83ms LinearSearch) validating high-dimensional design. Query: Vector ID=1, threshold=0.3.

Key Findings - ColorHistogram 32D:

LSH Search Dominance: LSH achieves $2.9\times$ faster search at 50K scale (1.67ms vs LinearSearch 4.83ms) and maintains advantage across all scales. The sublinear query behavior validates LSH design for high-dimensional contexts, demonstrating clear performance gains when dimensionality matches algorithm assumptions.

Insertion Trade-off: LSH insertion overhead (88.7ms at 50K) is $5.8\times$ higher than LinearSearch (15.2ms) due to random projection computation and multi-table indexing. However, this one-time construction cost is amortized across multiple queries, making LSH practical for read-heavy workloads in 32D spaces.

M-Tree Baseline Performance: M-Tree simplified implementation performs similarly to LinearSearch baseline, confirming that metric tree advantages require full implementation with covering radius optimization and balanced splitting—beyond scope of this validation experiment.

3.2 RGB 3D Validation - Low-Dimensional Comparison

Table 2 presents RGB 3D validation results demonstrating bidirectional curse of dimensionality—high-dimensional structures impose overhead in low-dimensional contexts.

Table 2. RGB 3D Validation Results (26K images, averaged across two queries)

Structure	Insert (ms)	Search (ms)	Found	Recall
Linear Search	1.00	2.32	8380	100%
Octree	5.48	2.12	8380	100%
Quadtree	6.92	2.25	8380	100%
Hash Dynamic	4.14	2.15	8380	100%
LSH ($L=10, k=4$)	53.91	4.62	8039	95.9%
M-Tree	11.85	2.46	7650	91.3%

Classical structures (Octree, Quadtree) dominate in RGB 3D space with simpler implementations achieving better performance and precision. LSH shows $2.2\times$ search slowdown vs Octree. Threshold=40.0.

Key Performance Findings:

Insertion Performance: Linear Search achieves the fastest insertion across all scales (1.00ms for 26K images) through simple array append operations. Classical tree structures show moderate overhead (Octree: 5.48ms, Quadtree: 6.92ms). High-dimensional structures exhibit severe degradation: LSH insertion (53.91ms) is $53.9\times$ slower than Linear and $7.8\times$ slower than Quadtree, while M-Tree (11.85ms) is

11.9× slower than Linear. The LSH overhead stems from computing random projections and maintaining L=10 hash tables, revealing impractical construction costs for RGB applications.

Search Performance: Classical structures dominate with Octree achieving 2.12ms, Quadtree 2.25ms, and Hash Dynamic 2.15ms—all within 6% performance band. Linear Search (2.32ms) remains competitive despite $O(n)$ complexity, benefiting from cache-friendly sequential access. LSH search (4.62ms) is 2.18× slower than Octree despite theoretical sublinear query promises, invalidating its core value proposition. M-Tree (2.46ms) provides no advantage over simpler Octree while sacrificing precision.

Bidirectional Curse of Dimensionality: Results demonstrate that high-dimensional structures suffer not only in truly high-dimensional spaces (their original design motivation) but also impose unnecessary overhead in low-dimensional RGB 3D contexts. The curse operates bidirectionally—structures optimized for $d \geq 50$ dimensions introduce computational complexity that provides no benefit when $d=3$, revealing fundamental algorithm-dimensionality mismatch.

3.3 Precision Analysis

Table 3. Similar Images Found and Precision Analysis (Similarity threshold = 40.0)

Algorithm	1K	5K	10K	25K	26K
Linear Search	303	1483	3109	7992	8380
Hash Search	387	1907	3987	10203	10697
Hash Dynamic	303	1483	3109	7992	8380
Quadtree	303	1483	3109	7992	8380
Octree	303	1483	3109	7992	8380
LSH	288	1423	2997	7668	8039
M-Tree	282	1265	2694	7272	7650

*Values represent number of similar images found within threshold=40.0.
Linear Search serves as ground truth baseline for exact similarity matching.*

Precision Analysis Findings:

Exact Matching Cluster: Four classical structures (Linear Search, Hash Dynamic, Quadtree, Octree) achieve identical 100% precision with 8380 results for the 26K dataset. This validates correct threshold application and establishes classical spatial indexing as ground truth for RGB similarity search. The exact match across diverse algorithmic approaches confirms implementation correctness and demonstrates that simple geometric partitioning suffices for precise RGB similarity.

Hash Search Systematic Expansion: Consistently finds 1.28× more images (10697 vs 8380) due to dynamic cell sizing (cell=30.0) capturing neighboring color regions beyond exact threshold boundaries. This systematic 27.7% over-inclusion persists across all scales, suggesting intentional broader matching rather than algorithmic error—potentially valuable for exploratory similarity applications accepting relaxed precision.

LSH Approximation Trade-off: Achieves 95.9% recall (8039/8380 results), demonstrating the inherent precision-performance trade-off of probabilistic hashing. The 4.1% precision loss stems from random projection quantization and probabilistic bucket collisions. However, this precision

sacrifice provides no performance benefit—LSH search time (4.62ms) significantly exceeds exact classical methods (Octree: 2.12ms), invalidating LSH’s fundamental value proposition of trading precision for speed in RGB 3D space.

M-Tree Precision Anomaly: Achieves only 91.3% recall (7650/8380 results), contradicting M-Tree’s theoretical guarantee of exact metric search. The 8.7% precision loss likely stems from aggressive covering radius pruning that eliminates valid candidates near threshold boundaries. This precision degradation provides no compensating performance advantage (2.46ms vs 2.12ms Octree), rendering M-Tree unsuitable for applications requiring threshold compliance. The precision loss suggests implementation challenges in balancing covering radius optimization with search completeness.

4 Discussion

The experimental results comparing seven data structures reveal critical insights into the applicability of high-dimensional indexing methods to low-dimensional RGB similarity search, challenging assumptions about algorithmic sophistication universally translating to practical performance gains.

4.1 Algorithm Selection Guidelines

Production Systems (>10K images): Deploy Octree for optimal performance-precision balance (2.12ms search, 100% precision, 5.48ms insertion). Quadtree provides competitive alternative (2.25ms search, 6.92ms insertion). Hash Dynamic offers slightly faster search (2.15ms) with identical precision.

Moderate Datasets (1K-10K images): Linear Search (1.00ms insertion, 2.32ms search, 100% precision) provides excellent performance with minimal implementation complexity. The algorithmic simplicity and guaranteed precision outweigh marginal performance gains from sophisticated structures at this scale.

Avoid for RGB Applications: LSH (53.91ms insertion, 4.62ms search, 95.9% recall) and M-Tree (11.85ms insertion, 2.46ms search, 91.3% recall) provide no practical benefit over classical structures while sacrificing both performance and precision. These structures designed for high-dimensional spaces impose prohibitive overhead in RGB 3D.

4.2 Scalability and Theoretical Divergence

Classical structures demonstrate excellent scalability from 1K to 26K images. Octree insertion scales from 0.15ms to 5.48ms, with search scaling from 0.06ms to 2.12ms. LSH exhibits catastrophic failure: insertion scales from 2.34ms to 53.91ms, exceeding all classical structures. M-Tree insertion scales from 0.30ms to 11.85ms, providing no search advantage (2.46ms vs 2.12ms Octree) while losing 8.7% precision.

LSH theory promises $O(n^\rho)$ sublinear query time for $\rho < 1$ in high dimensions, yet 26K search (4.62ms) is 2.18× slower than Octree (2.12ms). The fundamental mismatch:

LSH design assumes $d \geq 50$ dimensions where random projections preserve distances; in RGB $d=3$, projection overhead dominates without benefit. M-Tree’s 91.3% recall contradicts its exact search guarantee, revealing challenges in covering radius computation when threshold=40.0 spans significant color space. Octree’s geometric pruning aligns naturally with RGB cube geometry, enabling effective subtree elimination.

5 Conclusion

This comprehensive empirical study challenges conventional wisdom about data structure selection for RGB image similarity search by comparing classical spatial indexing methods against modern high-dimensional structures. Our evaluation of seven distinct approaches across 1K to 26,179 real images from the Animals-10 Kaggle dataset reveals that specialized high-dimensional structures provide no benefit in RGB 3D space while imposing significant performance and precision penalties.

5.1 Key Research Contributions

Classical Structure Dominance: Octree achieves optimal performance, combining fastest search time (2.12ms for 26K images) with 100% precision (8380 exact results). Quadtree (2.25ms search) and Hash Dynamic (2.15ms search) provide competitive alternatives with identical precision. Simple geometric partitioning demonstrates that algorithmic sophistication does not guarantee practical superiority.

LSH Catastrophic Failure: Locality-Sensitive Hashing exhibits complete failure in RGB 3D: 53.91ms insertion ($53.9\times$ slower than Linear), 4.62ms search ($2.18\times$ slower than Octree), and 95.9% recall (4.1% precision loss). The algorithm-dimensionality mismatch—LSH optimizes for $d \geq 50$ dimensions where random projections preserve distances; in RGB $d=3$, projection overhead dominates without benefit—reveals critical limitations in applying high-dimensional methods to low-dimensional problems.

M-Tree Precision Sacrifice: M-Tree’s triangle inequality pruning, theoretically guaranteeing exact metric search, paradoxically achieves only 91.3% recall (8.7% precision loss, 7650/8380 results) while providing no search advantage (2.46ms vs 2.12ms Octree). The 11.85ms insertion overhead and precision degradation render M-Tree unsuitable for RGB similarity applications requiring threshold compliance. This precision loss contradicts M-Tree’s theoretical guarantees and suggests implementation challenges in balancing covering radius optimization with search completeness.

Bidirectional Curse of Dimensionality: Results demonstrate that high-dimensional indexing structures suffer bidirectionally—degrading not only in truly high-dimensional spaces (their original design motivation) but also imposing unnecessary complexity in low-dimensional spaces. RGB 3D represents a “sweet spot” where simple spatial partitioning (Octree, Quadtree) dominates both performance and precision. Structures optimized for curse-of-dimensionality mitigation ironically become victims

of reverse curse effects when applied outside their design context.

Linear Search Competitive Reality: Simple sequential scanning (1.00ms insertion, 2.32ms search, 100% precision) remains competitive at 26K scale, demonstrating that algorithmic sophistication does not guarantee practical superiority. Cache-friendly sequential access patterns often outperform pointer-chasing tree traversal for moderate datasets, emphasizing that implementation details and hardware characteristics significantly influence practical performance rankings.

5.2 Practical Implications and Future Work

Recommendations: Deploy Octree for RGB similarity systems with $>10K$ images (optimal performance-precision balance). Use Linear Search for $<1K$ datasets (implementation simplicity, guaranteed precision). Avoid LSH and M-Tree for RGB applications—their theoretical high-dimensional advantages translate to practical disadvantages in RGB 3D space.

Future Directions: (i) Systematic investigation to identify the dimensionality threshold where high-dimensional structures begin outperforming classical spatial indexing (empirical studies spanning $d=3$ RGB through $d=128$ SIFT to $d=512$ deep embeddings); (ii) LSH parameter optimization for low dimensions (investigating whether reduced L and k values decrease overhead while maintaining search quality); (iii) M-Tree precision investigation (analyzing covering radius computation and node splitting to restore exact search guarantees); (iv) Alternative color spaces (evaluating LAB, LUV for different spatial locality characteristics); (v) GPU acceleration potential (determining whether massive parallelism can overcome LSH’s sequential overhead).

Author Contributions

Luan Carrieiros: Project lead, Hash Dynamic, LSH implementation, OpenCV integration. **Diego Rocha:** Octree/Quadtree. **Iago Ribeiro:** Linear Search, theoretical analysis. **Bernardo Temponi:** Hash Search. **Arthur Moraes:** M-Tree, optimization, CRE-ATE→TEST→DESTROY pattern.

Code Availability

Complete source code available at: <https://github.com/LuanCarrieiros/PAA>

References

- [1] Indyk, P. and Motwani, R. (1998). Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing (STOC '98)*, pages 604–613. ACM, New York, NY, USA.
- [2] Ciaccia, P., Patella, M., and Zezula, P. (1997). M-tree: An efficient access method for similarity search

in metric spaces. In *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB '97)*, pages 426–435. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

- [3] Corrado, A. (2018). Animals-10. Dataset available at: <https://www.kaggle.com/datasets/alessiocorrado99/animals10>. Accessed: 2025-11-21.
- [4] Bradski, G. (2000). The OpenCV library. *Dr. Dobbs's Journal of Software Tools*, 120:122–125.
- [5] Lee, C.-H., Lu, H.-y., and Horng, J.-H. (2018). Color quantization by hierarchical octa-partition in RGB color space. In *2018 IEEE International Conference on Applied System Invention (ICASI)*, pages 147–150.
- [6] Saad, A.-M. H. Y., Abdullah, M. Z., Alduais, N. A. M. M., and Sa'ad, H. H. Y. (2020). Impact of spatial dynamic search with matching threshold strategy on fractal image compression algorithm performance: Study. *IEEE Access*, 8:52687–52699.
- [7] UCI Machine Learning Repository (1999). Corel Image Features Dataset. Available at: <https://archive.ics.uci.edu/dataset/119/corel+image+features>. Accessed on: 29 Nov. 2025.